

26년 경북대 AX Intensive 교육

2주차 – ChatGPT를 활용한 데이터분석



KNU KYUNGPOOK
NATIONAL UNIVERSITY



LG전자

소 속 : 경북대학교/인공지능학과

이 름 : 김현철 교수

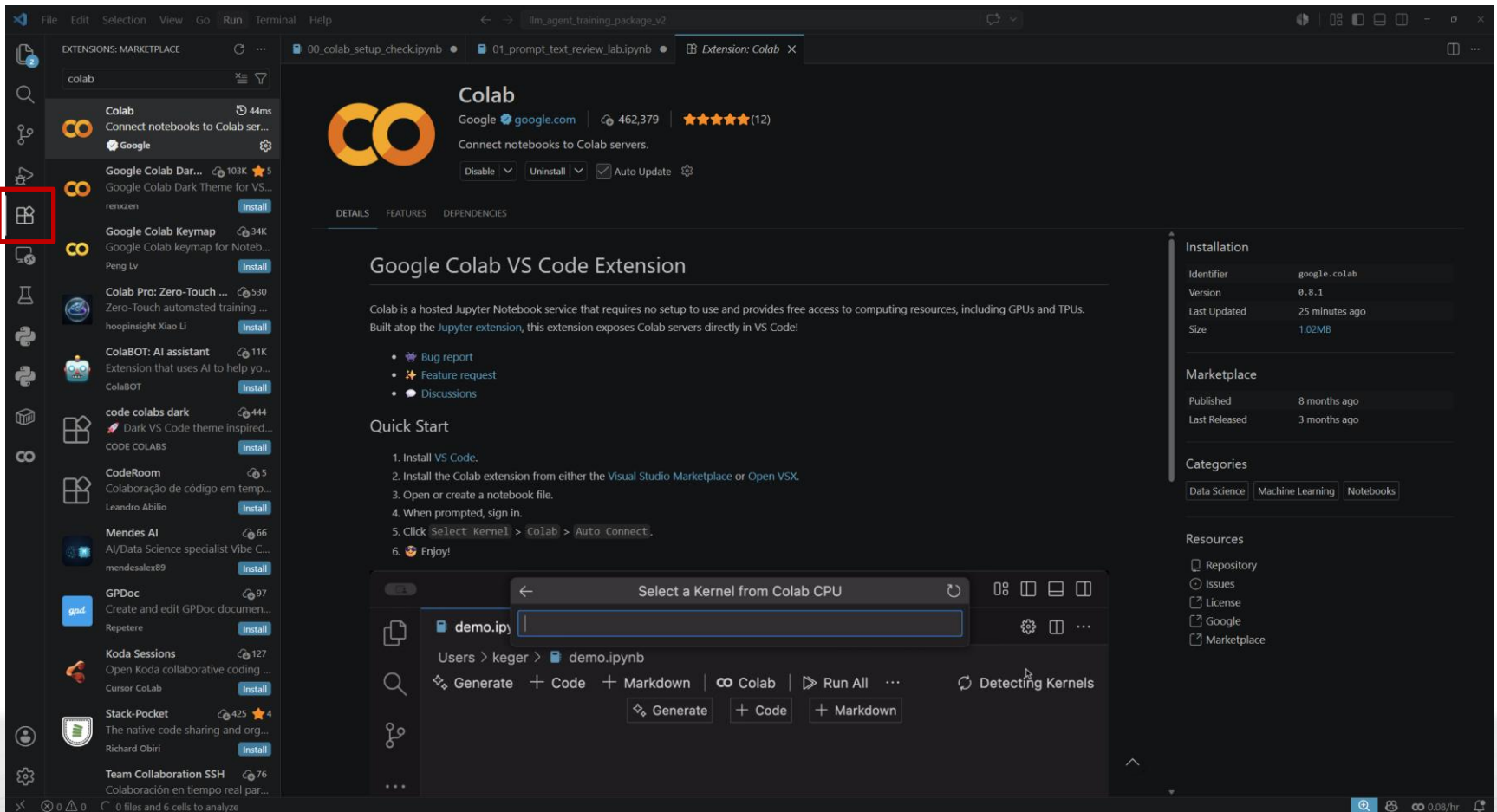
E-Mail : hyunchul_kim@knu.ac.kr

Timetable

시간표		학습 내용
1차시	09:00 – 09:50	PoC 구현을 위한 데이터 분석 및 ChatGPT를 활용한 데이터 분석 데이터 시각화 및 ChatGPT 기반 실습
2차시	10:00 – 10:50	
3차시	11:00 – 11:50	
Lunch Break (11:50 – 13:30)		
5차시	13:30 - 14:20	랩실 멘토링
6차시	14:30 – 15:20	
7차시	15:30 – 16:20	
8차시	16:20 – 17:30	

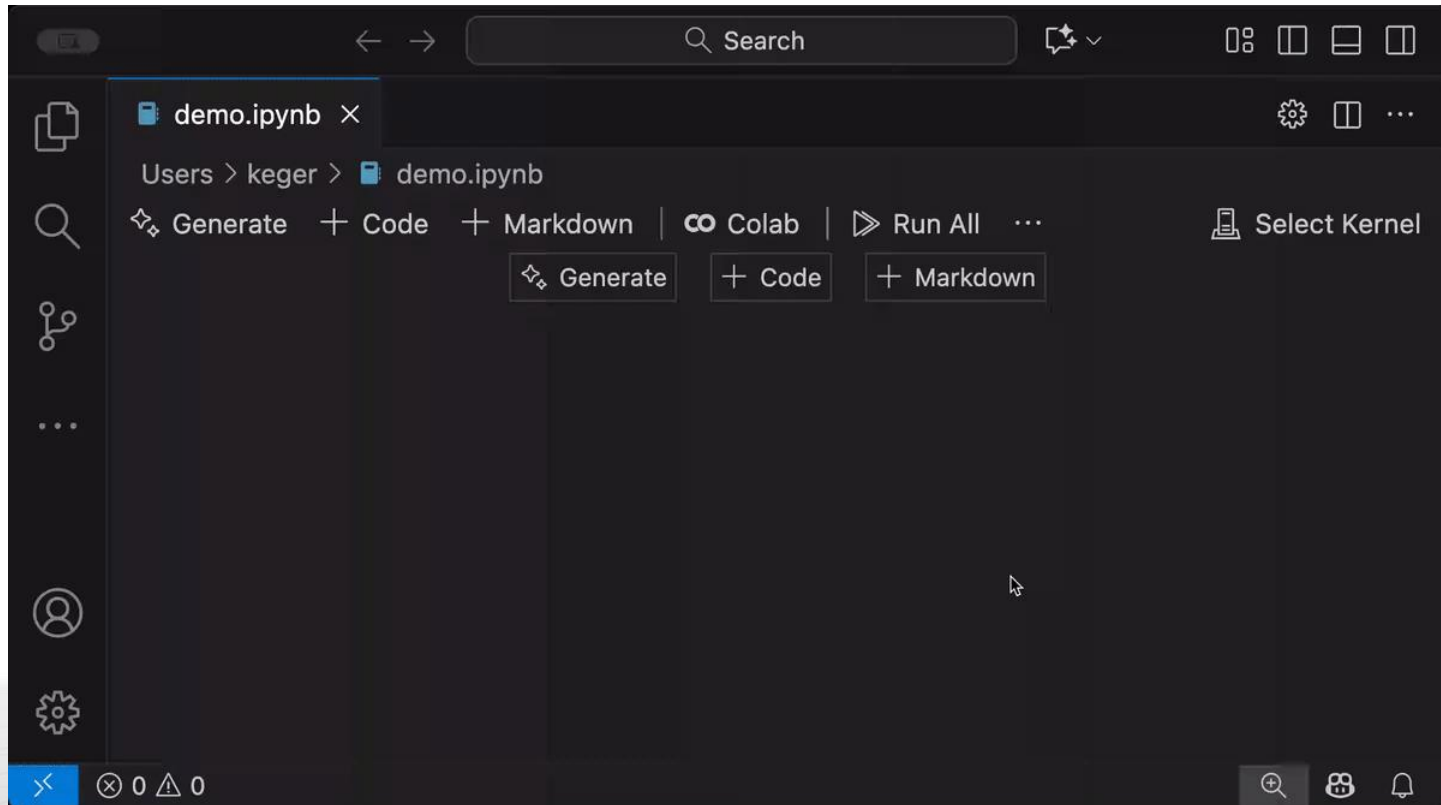
0. VScode setting

- Extension > Colab 설치



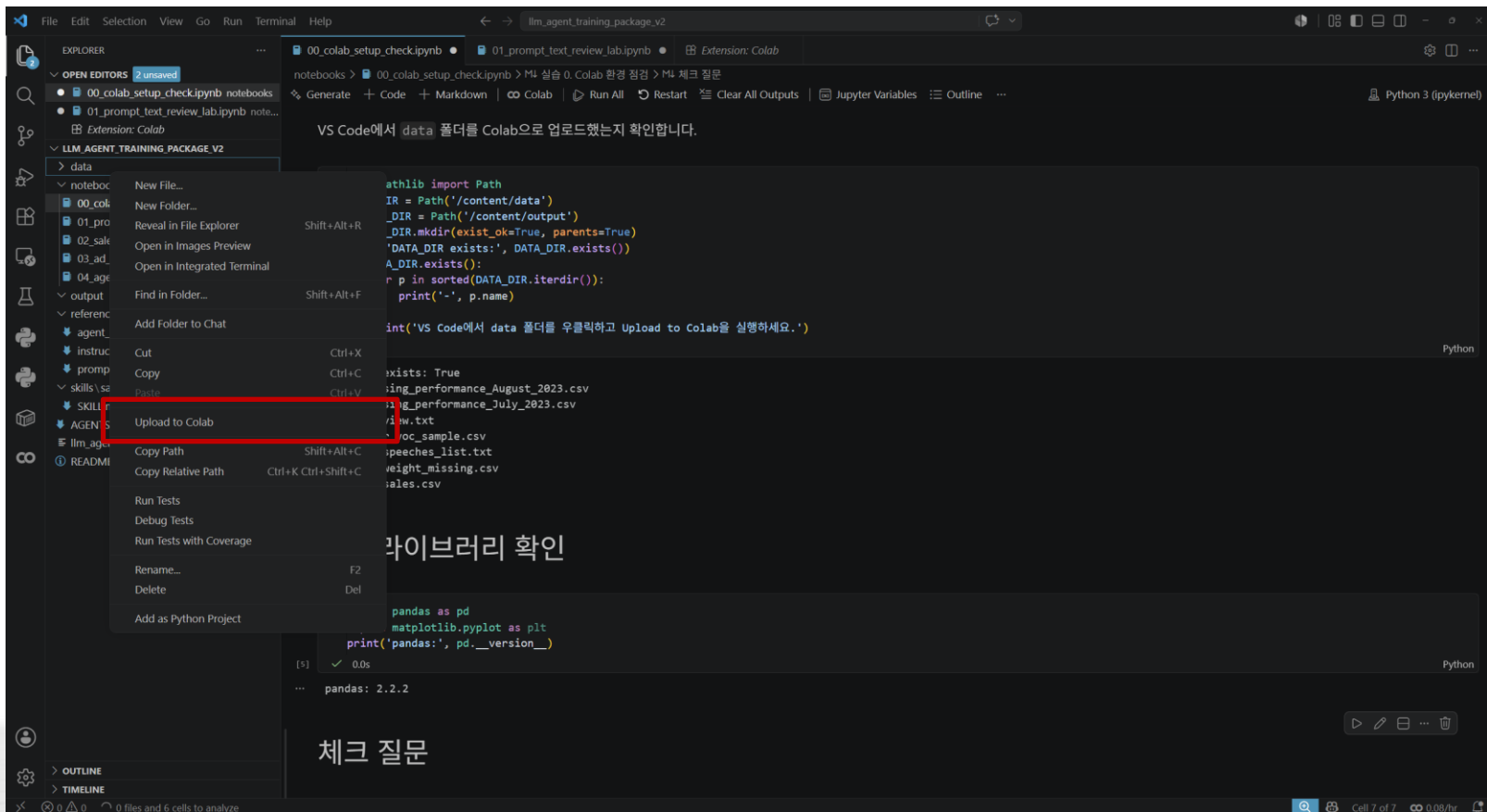
0. VScode setting

- 00_colab_setup_check.ipynb 열기
- Select Kernel > Colab > Auto Connect
- Google 계정 연결



0. VScode setting

- Data를 colab에 업로드
- 데이터 경로 확인 (/content/data)



0. LLM, ChatGPT, Agent

- LLM (Large Language Model)
 - 대규모 텍스트 데이터로 학습한 생성 모델
- ChatGPT
 - OpenAI의 LLM을 사용자가 쉽게 이용할 수 있도록 만든 대화형 서비스
- Agent
 - 사용자가 정한 목표를 달성하기 위해 필요한 작업을 판단하고, 도구를 사용하며, 여러 단계를 반복해서 수행하는 AI 시스템

LLM

- 문서 요약
- 문장 교정
- 번역
- 데이터 해석
- 정보 분류 및 정리

ChatGPT

- LLM
- + 채팅화면
- + 대화 기록
- + 파일 업로드
- + 외부 도구 사용

Agent

- 사용자의 목표 설정
- → 작업 계획 수립
- → 파일 확인
- → 도구 선택 및 실행
- → 결과 확인
- → 검증 및 보고

0. 용어들

- Token
 - AI가 글을 처리하는 최소 단위. 단어보다 작을 수도, 클 수도 있음.
 - 영어는 약 4글자가 1토큰, 한국어는 글자당 더 많은 토큰 사용.
 - ChatGPT 등의 요금과 처리량이 토큰 단위로 계산됨.
- Context Window
 - 모델이 한 번에 “기억”할 수 있는 토큰의 양.
 - 대화가 길어지면 이 창을 넘지 않도록 관리해야 함. (압축)
- Reasoning effort
 - 모델이 답하기 전에 “얼마나 깊게 생각할지” 결정
 - Minimal ~ xhigh (low: 빠르고 저렴, high: 느리지만 똑똑)
- Prompt
 - AI에게 주는 지시문.
- Thread
 - 하나의 작업 세션. 프롬프트 + AI 응답 + 도구 호출

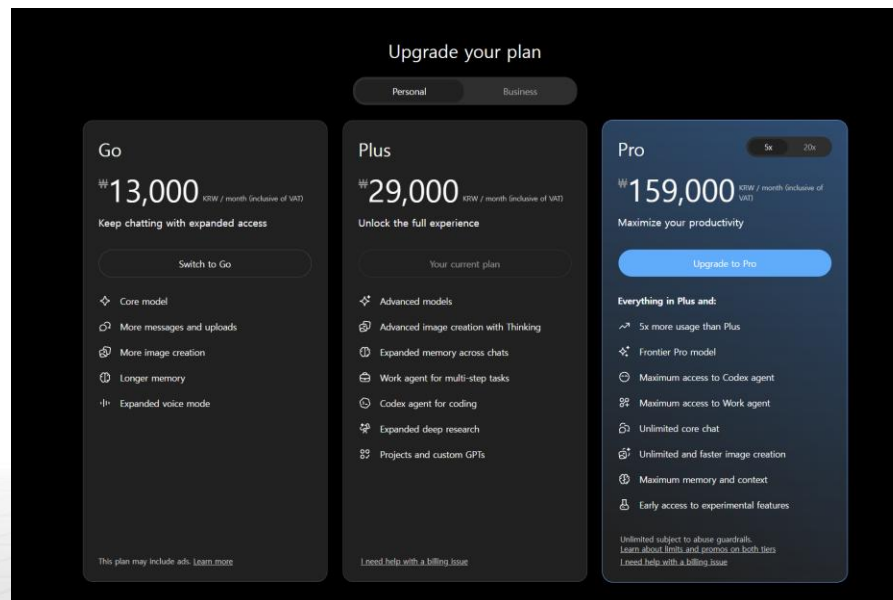
1. ChatGPT 데이터 분석

- 개요

- 챗GPT(ChatGPT) 데이터 분석은 챗GPT가 파이썬 코드를 직접 실행해서 결과를 돌려주는 기능

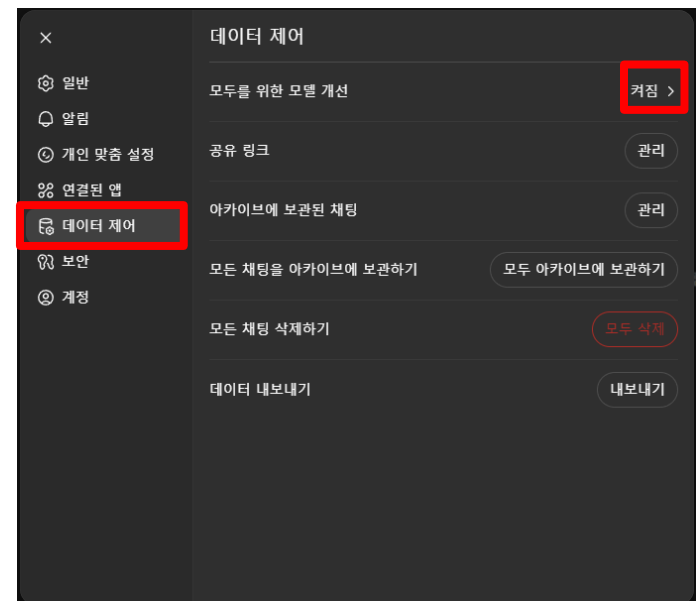
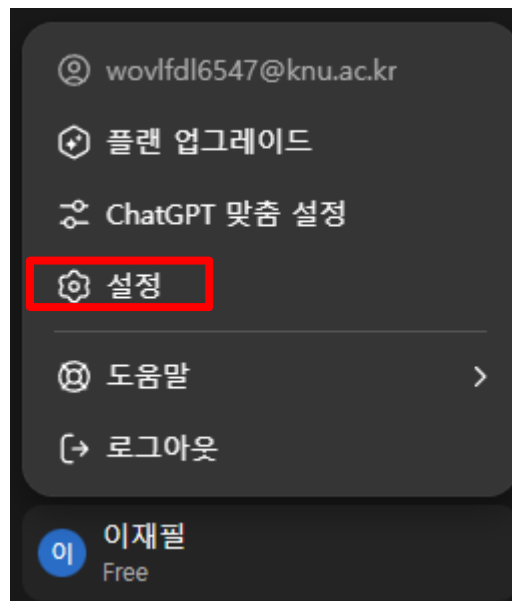
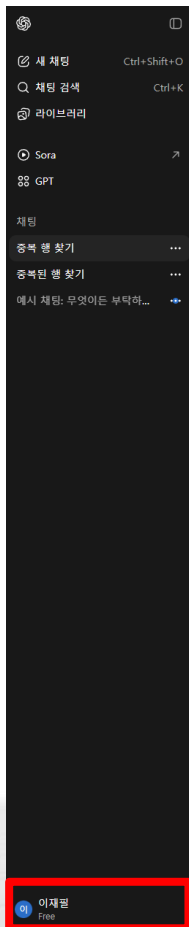
- 데이터 분석 실행 방법

- 무료 플랜: GPT-5.5 Instant를 제한적으로 이용
- Plus 유료: GPT-5.6 고급 추론 모델 및 심층 리서치 기능 이용



1. ChatGPT 데이터 분석

- 설정
 - 중요한 데이터(회사자료, 개인신상 등)를 지키기 위한 설정



1. ChatGPT 데이터 분석

- LLM이 잘 하는 일과 주의할 일

잘하는 일

- 문서 요약·초안 작성
- 분류·정리·표 변환
- 아이디어 도출
- 반복 형식 변환
- 데이터 탐색 보조

주의할 일

- 환각(Hallucination)
- 최신 사실 확인
- 사내 규정 판단
- 법률·재무·인사 최종 결정
- 민감 데이터 처리

1. ChatGPT 데이터 분석

- ChatGPT model

- 모델을 고르는 3가지 기준

- 품질 ↔ 속도 ↔ 비용 (Trade-off)

- 예시

- 품질이 최우선 (아키텍처 설계, 까다로운 버그) → gpt-5.5

- 균형 (대부분의 일상 작업) → gpt-5.4

- 속도·비용 우선 (반복 수정, 파일 훑기) → gpt-5.4-mini

- 즉답 페어코딩 → gpt-5.3-codex-spark

- 추론 강도도 같이 선택

- 적당한 모델 + 높은 추론이 나온 경우가 많음

모델	성격	추천 용도	비용/속도
gpt-5.5	플래그십, 최강 추론	복잡한 코딩, 설계, 리서치	비싸고 느림(깊음)
gpt-5.4	주력 프런티어, 전 플랫폼 호환	일상적인 전문 작업	균형
gpt-5.4-mini	경량·고속	대용량 스캔, 단순 작업, 서브에이전트	싸고 빠름
gpt-5.3-codex-spark	실시간 프리뷰(텍스트 전용)	즉각적인 코딩 반복(ChatGPT Pro 전용)	매우 빠름

1. ChatGPT 데이터 분석

- 프롬프트 엔지니어링 = 업무 지시서 작성법

역할	목적	자료
해야 할 일	조건	출력 형식

예시

“당신은 영업기획 담당자를 지원하는 데이터 분석가입니다.
경영회의에서 사용할 최근 매출 흐름을 분석해 주세요.
첨부한 CSV를 사용하고, 월별 매출·지역별 목표 달성률·이상치를 확인하세요.
확인되지 않는 원인은 추측하지 말고, 결론 3개와 근거 표로 작성해 주세요.”

1. ChatGPT 데이터 분석

• ChatGPT vs API vs Codex

구분	무엇인가	누가 쓰나
ChatGPT	웹/앱의 대화 인터페이스	모두 — 질문, 글쓰기, 아이디어
API	코드로 GPT를 호출하는 인터페이스	개발자 — 내 앱에 AI 기능 탑재
Codex	GPT를 코딩 에이전트로 만든 도구	개발/제작 — 파일 편집·실행·자동화

- ChatGPT: 채팅 상담원. 답을 주지만 컴퓨터를 직접 만지진 않음.
 - 단순 질의 응답 시 사용.
 - Ex) "~~이 뭐야?", "이 에러 메시지 무슨 뜻이야?"
- API: 상담원을 내 프로그램 안에 부품으로 사용.
 - 기능을 서비스에 넣을 때 사용.
 - Ex) 내가 만든 웹사이트에 AI 우선순위 추천 등의 기능 추가 시
- Codex: 상담원에게 내 작업 폴더 권한 부여. 직접 파일 수정 가능.
 - 버그 수정, 내 폴더의 자료 확인 후 보고서/테스트 작성 등
 - Ex) "document에 있는 자료 보고 웹사이트로 만들어줘"

1. 프롬프트 개선

- 목표: 모호한 요청을 좋은 업무 요청으로 바꾸기

Before

"매출 데이터 좀 봐줘."



After

역할, 목적, 분석 항목, 검증 기준, 출력 형식을 포함한 업무 요청으로 변환합니다.

Ex)

목적: 어디에 쓸 결과인가?
자료: 어떤 파일/컬럼을 쓰는가?
조건: 추측 금지·계산 검증 등
출력: 요약, 표, 차트, 제안

1. 텍스트 분석

- 중복 행 찾기
 - 수업보조자료 중 famous_speeches_list.txt 파일을 프롬프트 창에 입력 후
 - "중복된 행이 있는지 찾아주세요" 라고 질문시

파일에서 중복된 행(같은 연설 제목을 가진 항목)은 다음과 같습니다:

1. "Gettysburg Address" - 에이브러햄 링컨, 1863년, 게티즈버그, 미국
"The Gettysburg Address" - 에이브러햄 링컨, 1863년, 게티즈버그, 미국
→ 동일한 연설을 다른 이름으로 두 번 기록.
2. "I Have Been to the Mountaintop" - 마틴 루터 킹 주니어, 1968년, 멤피스, 미국
"I've Been to the Mountaintop" - 마틴 루터 킹 주니어, 1968년, 멤피스, 미국
→ 같은 연설 제목의 표기 차이.
3. "Checkers Speech" - 리처드 닉슨, 1952년, 로스앤젤레스, 미국
"The Checkers Speech" - 리처드 닉슨, 1952년, 로스앤젤레스, 미국
→ "The" 유무 차이.
4. "Remarks at the Brandenburg Gate" - 로널드 레이건, 1987년, 베를린, 독일
"The Speech at the Brandenburg Gate" - 로널드 레이건, 1987년, 베를린, 독일
→ 같은 연설을 다르게 표기.
5. "Speech on the Vietnam War" - 린든 B. 존슨, 1965년, 워싱턴 D.C., 미국
"The Vietnam War Speech" - 린든 B. 존슨, 1965년, 워싱턴 D.C., 미국
→ 동일 연설의 명칭 차이.

필요하다면, 중복을 제거한 정리된 목록 파일을 만들어 드릴까요?

1. 텍스트 분석

• 중복 행 찾기

- “사람 이름과 연도를 키로해서 중복이 있는지 확인해주세요”를 프롬프트 창에 입력

famous_speeches_list.txt 에서 사람 이름 + 연도를 기준으로 중복을 확인한 결과, 다음과 같은 중복이 있습니다:

중복된 항목 (이름 + 연도 동일)

1. 마틴 루터 킹 주니어, 1968년

- "I Have Been to the Mountaintop"
- "I've Been to the Mountaintop"
- "I Am a Man"

2. 에이브러햄 링컨, 1863년

- "Gettysburg Address"
- "The Gettysburg Address"

3. 리처드 닉슨, 1952년

- "Checkers Speech"
- "The Checkers Speech"

4. 로널드 레이건, 1987년

- "Remarks at the Brandenburg Gate"
- "The Speech at the Brandenburg Gate"

5. 린든 B. 존슨, 1965년

- "Speech on the Vietnam War"
- "The Vietnam War Speech"
- "The American Promise"
- "We Shall Overcome"

원한다면 이름+연도 기준으로 중복 제거한 새로운 파일을 만들어 드릴까요?

그렇게 해줘

이름과 연도를 기준으로 중복을 제거한 파일을 만들었습니다.

여기서 다운로드하세요. <>

📄 📁 🔍 🔗 ⌂ ↻

1. 텍스트 분석

• 서평 분석

- 수업보조자료 중 book_review.txt 파일을 프롬프트 창에 입력 후 “리뷰 분석 해줘” 라고 질문 시

업로드하신 book_review.txt 파일을 분석한 결과는 다음과 같습니다:

1. 리뷰 요약

- 긍정적 리뷰 (2개)
 - 교보문고, 평점 4/4
 - “실전 비전 머신러닝에 도움이 되네요”
 - “컴퓨터에 대한 근본적 지식”
 - 이 두 리뷰는 책의 실전적인 도움과 기초 지식 전달에 대해 긍정적으로 평가하고 있습니다.
- 부정적 리뷰 (1개)
 - 알라딘, 평점 1/5
 - 불만 내용: 책의 실습 코드와 깃허브 코드의 불일치, 설명 부족, 생략된 부분이 많다는 점.
 - 리뷰어는 책의 내용 구성 및 코드 품질에 큰 실망을 표출.

2. 평점 분석

- 교보문고: 4/4 (2개)
- 알라딘: 1/5 (1개)
- 평균 평점(단순 계산):
 - 교보문고 리뷰는 4점 만점, 알라딘은 5점 만점하므로 직접 비교는 어렵습니다.
 - 변환 후 평균(5점 환산 기준):
 - 교보문고: $(4/4 = 5/5) \rightarrow 5$ 점
 - 알라딘: $1/5 \rightarrow 1$ 점
 - 평균 $\approx (5 + 5 + 1) / 3 = 3.67$ 점

3. 감성 분석

- 긍정적 키워드: “도움”, “근본적 지식”
- 부정적 키워드: “최악”, “시간이 아까워요”, “다르냐고요”

다음으로 리뷰 데이터를 **긍정/부정 비율 시각화(파이 차트)**로 만들어드릴까요?

📄 🗂 🔍 🔊 🎨 🔄 🏠

1. 텍스트 분석

• 서평 분석

- “평점 기준이 4점 만점도 있고 5점 만점도 있어 일관적이지 않다. 10점 만점으로 통일해줘” 라고 추가 분석 요청
- 이후 “평균평점은 얼마지?” 라는 질문을 통해 평균평점에 대한 내용 요청



알겠습니다! 각 책의 서평을 10점 만점 기준으로 재평가해 보겠습니다.

1. 교보문고, 실전 비전 머신러닝에 도움이 되네요

- 원래 평점: 4/4
- 10점 만점으로 환산: $\frac{4}{4} \times 10 = 10$ 점

2. 교보문고, 컴퓨비전에 대한 근본적 지식

- 원래 평점: 4/4
- 10점 만점으로 환산: $\frac{4}{4} \times 10 = 10$ 점

3. 알라딘, 진짜 뭘 위한 책일까요

- 원래 평점: 1/5
- 10점 만점으로 환산: $\frac{1}{5} \times 10 = 2$ 점

따라서 새로운 평가 결과는 다음과 같습니다:

- 교보문고, 실전 비전 머신러닝에 도움이 되네요: 10/10
- 교보문고, 컴퓨비전에 대한 근본적 지식: 10/10
- 알라딘, 진짜 뭘 위한 책일까요: 2/10



주어진 리뷰들을 바탕으로 평균 평점을 계산해 보겠습니다.

1. 교보문고, 실전 비전 머신러닝에 도움이 되네요: 10/10
2. 교보문고, 컴퓨비전에 대한 근본적 지식: 10/10
3. 알라딘, 진짜 뭘 위한 책일까요: 2/10

평균 평점은 각 평점을 더한 후 총 리뷰의 수로 나누어 계산합니다.

$$\text{평균 평점} = \frac{\text{총 평점 합}}{\text{총 리뷰 수}}$$

$$\text{총 평점 합: } 10 + 10 + 2 = 22$$

$$\text{총 리뷰 수: } 3$$

$$\text{평균 평점} = \frac{22}{3} \approx 7.33$$

따라서, 주어진 리뷰들의 평균 평점은 약 7.33점입니다.

1. 텍스트 분석

• 서평 분석

- 평점이 극단 적인 이유를 분석하기 위해 “개별 평점이 왜 중간이 없고 극단적일까?” 라고 질문

업로드된 리뷰 데이터를 보면, 평점이 10점(최고점) 혹은 **2점(매우 낮음)**으로 극단적인 이유는 다음과 같이 해석할 수 있습니다:

1. 기대치와 만족도의 양극화

- 긍정적 리뷰(10점): 실용성과 기초 지식 제공이 기대에 부합했을 때 높은 점수를 주는 경향이 있습니다.
- 부정적 리뷰(2점): 코드 불일치나 설명 부족 등 기본 신뢰가 깨진 경우, 실망감이 크기 때문에 매우 낮은 점수를 주게 됩니다.

2. 온라인 리뷰의 특성

- 중간 정도 만족한 독자들은 굳이 리뷰를 남기지 않는 경우가 많습니다.
- ***매우 만족**하거나 **매우 불만***을 가진 사용자들이 리뷰를 남기는 비율이 높아, 데이터가 양극화됩니다.

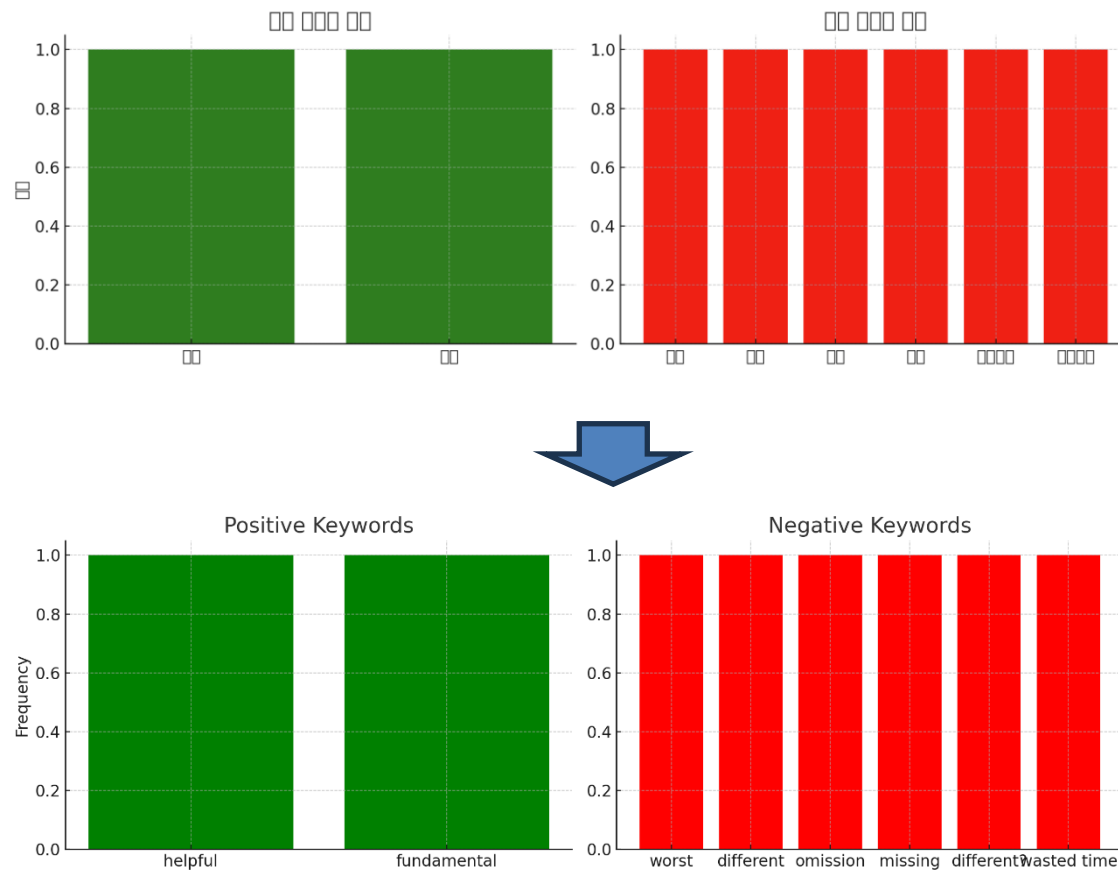
3. 책의 특성

- 기술서적, 특히 실습서의 경우 코드 오류나 불일치가 있으면 책 자체의 가치가 크게 떨어집니다.
- 반대로 코드와 내용이 잘 맞아떨어지면 실무에서 유용하다고 평가받아 최고점을 줄 가능성이 큼니다.

1. 텍스트 분석

• 서평 분석

- 분석에 대한 시각화를 위해 “리뷰 텍스트에서 긍정/부정 키워드 빈도를 분석해서 극단적인 평가 원인을 더 구체적으로 시각화해 줘” 라고 요청



1. 텍스트 분석

- Top K와 Top P 설명
 - "I ate pizza while it was still" 뒤에 나올 수 있는 단어들 각각에 대해 확률을 알고 싶다. 라고 질문시

맞습니다. "I ate pizza while it was still" 다음에 올 수 있는 단어들은, 확률적으로 가장 높은 몇 개를 다음과 같이 예측할 수 있습니다. 예시 확률은 영어 언어 모델(GPT-계열, BERT-like 모델 기반)을 가정한 수치입니다.

다음 단어 예측

문장: "I ate pizza while it was still _"

다음 단어 후보	확률 예시
warm	30%
hot	25%
warmish	10%
cold	5%
marinating	3%
good	2%
기타 (나머지)	25%

- **warm, hot** 같은 단어가 가장 자연스럽게 자주 사용되므로 높은 점수를 받습니다.
- **cold**는 역설적으로 낮은 가능성이지만 문맥에 따라 일부 발생할 수 있습니다.
- **marinating** 등 덜 일반적인 단어는 확률이 매우 낮습니다.
- 표의 "기타" 항목은 수많은 나머지 단어가 누적된 결과입니다.

왜 이런 결과가 나올까?

언어 모델은 이전 문맥과 훈련 데이터 통계를 바탕으로 다음 단어의 확률 분포를 계산합니다 towardsai.net en.wikipedia.org . 예를 들면:

- "while it was still warm/hot" 같은 구문이 자주 등장하면, 모델은 해당 단어에 높은 확률을 할당합니다.
- 반면 **cold**는 문맥상 어색하므로 낮은 확률을 가지며, **marinating**처럼 동사 변형도 드물기 때문에 거의 발생하지 않습니다.

모델이 단 하나의 다음 단어만 예측하는 것이 아니라, 가능한 단어들에 대해 확률 분포를 계산하며, 그 중 상위 몇 개를 예시로 드린 것입니다.

추가 정보

- 이 확률 값은 실제 수치가 아닌 **예시** 수준입니다. 실제 언어 모델(GPT-3/4, BERT-based LM)은 훈련 데이터와 크기에 따라 다르게 분포합니다.
- 더 정확한 수치를 원하시면, 직접 모델에 프롬프트를 주고 **logits** → **softmax**를 통해 다음 단어 분포를 확인할 수 있습니다.

원하신다면, 간단한 Python 예제 코드를 제공해 드릴까요?

📄 📌 🔍 🔊 ↺ ↻ 🔗 🌐 출처

1. 텍스트 분석

- Top K와 Top P 설명
 - Top K와 Top P에 대한 설명을 듣기 위해서 이전의 답변을 활용하여 GPT에 질문을 한다.
 - 질문 예시
 - warm: 약 30%
 - hot: 약 25%
 - warmish: 약 10%
 - cold: 약 5%
 - marinating: 약 3%
 - good: 약 2%
 - 기타 (나머지) 25%

이 대답에서 Top K와 Top P의 여러 조합을 예로 들어서 설명해줘

1. 텍스트 분석

- Top K와 Top P 설명
 - Top K / P에 대한 답변

1. Top-K 샘플링 예시

Top-K는 가장 확률이 높은 K개의 단어만 남기고 나머지를 제거한 뒤 확률을 재정규화합니다.

예시: K=3

- 상위 3개 단어: warm (30%), hot (25%), warmish (10%)
- 나머지(cold, marinating, good, 기타)는 제거.
- 정규화 후 확률:
 - warm: $\frac{30}{30+25+10} \approx 42.9\%$
 - hot: $\frac{25}{30+25+10} \approx 35.7\%$
 - warmish: $\frac{10}{30+25+10} \approx 14.3\%$

예시: K=5

- 상위 5개 단어: warm, hot, warmish, cold, marinating
- 정규화 후 확률:
 - warm: $\frac{30}{30+25+10+5+3} \approx 36.6\%$
 - hot: $\frac{25}{73} \approx 34.2\%$
 - warmish: $\frac{10}{73} \approx 13.7\%$
 - cold: $\frac{5}{73} \approx 6.8\%$
 - marinating: $\frac{3}{73} \approx 4.1\%$

2. Top-P (Nucleus Sampling) 예시

Top-P는 확률 누적합이 P 이상이 되는 최소 단어 세트를 선택합니다.

예시: P=0.7

- 확률 높은 순서대로 누적:
 - warm (30%) → 0.30
 - hot (25%) → 0.55
 - warmish (10%) → 0.65
 - cold (5%) → 0.70 → 여기서 멈춤.
- 남은 단어: warm, hot, warmish, cold
- 정규화 후 확률:
 - warm: $\frac{30}{30+25+10+5} = 37.5\%$
 - hot: $\frac{25}{70} = 35.7\%$
 - warmish: $\frac{10}{70} = 14.3\%$
 - cold: $\frac{5}{70} = 7.1\%$

예시: P=0.9

- 누적: warm (30%) → hot (25%) → warmish (10%) → cold (5%) → marinating (3%) → good (2%) = 누적 75%.
기타에서 15% 추가하면 90% 이상. 따라서 기타 포함.
- 이 경우 사실상 전체 단어가 포함됩니다.

1. 텍스트 분석

- Top K와 Top P 설명
 - 숫자에 약한 어린이가 이해해야하는 상황
 - 직관적 이해를 돕는 예시를 부탁
 - “지금 숫자가 너무 많아서, 좀더 직관적으로 이해할 수 있게 설명해줄래? 라고 질문

Top-K (상위 K개만 남기기)

- 비유: "인기 투표에서 상위 몇 명만 후보로 남기는 것"
- 예를 들어, K=3이면 warm, hot, warmish 3개 후보만 남기고 나머지는 버립니다.
- 마치 인기순위 TOP3만 놓고 다시 투표하는 것과 같아요.

Top-P (확률 누적합으로 선택하기)

- 비유: "점유율이 70%에 도달할 때까지 인기 후보를 모으는 것"
- P=0.7이라면 warm(30%) + hot(25%) + warmish(10%) + cold(5%) = 70%
→ 여기까지만 후보로 사용하고 나머지는 제외.
- 확률 합이 기준(P)에 도달할 때까지 단어를 유동적으로 뽑습니다.

차이 간단 요약

- Top-K는 무조건 "상위 K개만 본다."
- Top-P는 "확률 누적이 P(예:70%)가 될 때까지 뽑는다."
(상황에 따라 2개, 4개 등 후보 개수가 달라짐)

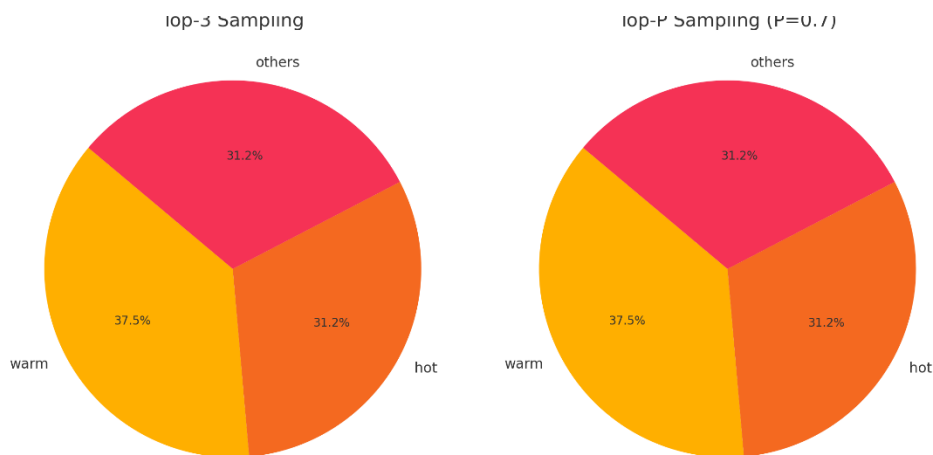
직관적 예시

당신이 피자 토핑을 고른다고 해봅시다:

- Top-3 (K=3): 항상 치즈, 페퍼로니, 베이컨만 남겨 놓고 고릅니다.
- Top-70% (P=0.7): 인기 있는 토핑부터 점유율이 70%를 넘을 때까지 치즈(30%), 페퍼로니(25%), 베이컨(10%), 올리브(5%) → 여기까지!

1. 텍스트 분석

- Top K와 Top P 설명
 - 시각화를 위한 요청
 - “그림(파이차트)으로 만들어서 Top-K와 Top-P가 어떻게 다른지 한눈에 볼 수 있게 시각화해줄래?”



잘못된 답을 줄 수 도 있으므로, 항상 체크하여야한다!

1. 텍스트 분석

- Top K와 Top P 설명
 - Top K, Top P 의 조합에 대해서 알고 싶은 경우
 - “Top K와 Top P를 조합한 예시를 보여줘” 라고 질문 시

가정한 확률 분포

makefile

복사 편집

warm: 30%

hot: 25%

warmish: 10%

cold: 5%

marinating: 3%

good: 2%

others: 25%

총합 = 100%

예시 1: Top-K=5 + Top-P=0.7

1. Top-K=5: 상위 5개 단어만 남깁니다.

warm, hot, warmish, cold, marinating (나머지 제외).

2. Top-P=0.7 (70%): 남은 5개에서 확률 누적합 70%가 될 때까지 선택합니다.

- warm(30%) → hot(25%) → warmish(10%) → cold(5%) = 70% → 여기서 멈춤.

3. 최종 후보: warm, hot, warmish, cold

정규화: warm(42.9%), hot(35.7%), warmish(14.3%), cold(7.1%).

예시 1: Top-K=5 + Top-P=0.7

1. Top-K=5: 상위 5개 단어만 남깁니다.

warm, hot, warmish, cold, marinating (나머지 제외).

2. Top-P=0.7 (70%): 남은 5개에서 확률 누적합 70%가 될 때까지 선택합니다.

- warm(30%) → hot(25%) → warmish(10%) → cold(5%) = 70% → 여기서 멈춤.

3. 최종 후보: warm, hot, warmish, cold

정규화: warm(42.9%), hot(35.7%), warmish(14.3%), cold(7.1%).

예시 2: Top-K=3 + Top-P=0.5

1. Top-K=3: warm, hot, warmish 만 남김.

2. Top-P=0.5: warm(30%) → hot(25%) = 55% (50% 초과) → 여기서 멈춤.

3. 최종 후보: warm, hot

정규화: warm(54.5%), hot(45.5%).

비유

- Top-K는 "상위 K개 단어만 보겠다."
- Top-P는 "확률 합이 P를 넘을 때까지만 후보를 보겠다."
- 둘을 같이 쓰면 → "상위 K개 중에서, 확률 합이 P 이상이 될 때까지만 선택."

1. 텍스트 분석

- Top K와 Top P 설명
 - 추가적으로 “Top K=0 & Top P=0.5 로 설정한다면?” 라는 질문의 답변
 - 답변에서 “누적 확률이 0.5 이내인 단어는 “hot”과 “warm”입니다.” 라는 내용은 틀린 내용으로, 무료버전의 GPT분석 시 꼭 답변을 확인하는 과정이 필요하다.



Top-K=0 및 Top-P=0.5로 설정할 경우, 이는 Top-P(Nucleus) 샘플링에서의 특정 조건을 나타냅니다. 이 설정은 다음과 같이 해석됩니다:

- **Top-K=0:** 후보 단어를 K개 선정하는 것이 아니라, 모든 가능한 단어를 고려합니다.
- **Top-P=0.5:** Nucleus 샘플링에서는 누적 확률이 0.5 이내인 단어들 중에서 선택됩니다.

이 설정을 바탕으로 예시를 설명해 보겠습니다.

주어진 확률 분포는:

- hot: 50%
- warm: 20%
- fresh: 15%
- crispy: 10%
- cheesy: 5%

예시: Top-K=0 & Top-P=0.5 샘플링

1. 누적 확률 계산:

- hot: 50%
- warm: 20%

누적 확률이 0.5 이내인 단어는 "hot"과 "warm"입니다.

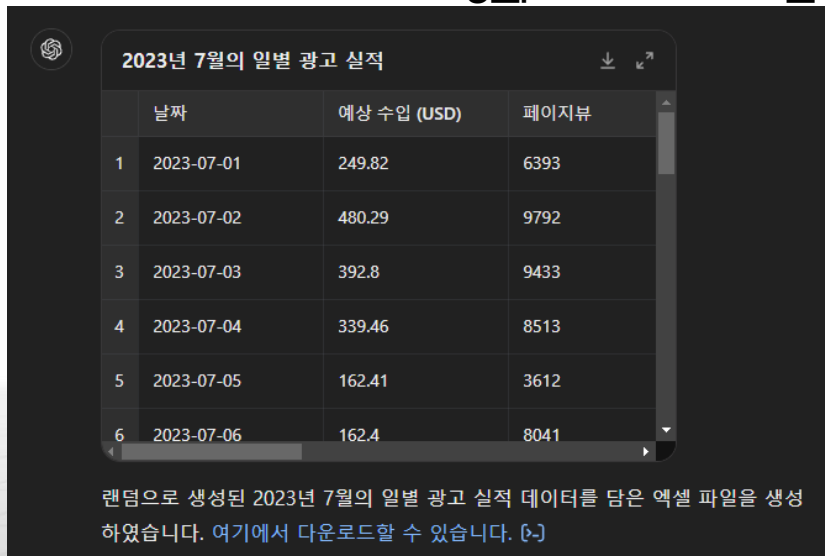
2. 선택:

- Top-P=0.5 설정에서는 누적 확률이 0.5 이내인 단어들 중에서 확률에 따라 선택이 이루어 집니다. 따라서 "hot"과 "warm" 중에서 확률에 따라 선택될 것입니다. 주어진 예시에서는 "hot"이 50%의 확률을 가지고 있으므로, "hot"이 선택될 가능성이 더 높습니다.

이렇게 설정된 경우에는 Top-P 샘플링 기법에 따라 누적 확률이 0.5 이내인 단어들 중에서 선택이 이루어지며, Top-K 값이 0이므로 K 개수에 대한 제한은 없습니다.

2. 엑셀 및 구글 스프레드시트 분석

- 데이터 분석 실습을 위한 데이터 생성
 - GPT 프롬프트 창에 아래와 같은 글을 입력
 - 데이터 분석 실습을 위해 엑셀 파일을 만들려고 한다. 이 파일은 2023년 7월의 일별 광고 실적을 담고 있으며 다음과 같은 열을 갖는다. 1.날짜 2. 예상 수입 (USD) 3. 페이지뷰 4. 페이지 RPM (USD) 5. 노출수 6. 노출 RPM (USD) 7. 조회 가능 Active View 8. 클릭수 범위는 2023년 7월 1일부터 7월 31일까지이다. 랜덤으로 엑셀 파일을 만들어줘
 - GPT는 아래와 같은 답변과 함께 다운로드 링크를 반환
 - 보조자료중 advertising_performance_July_2023.xlsx 파일로 저장



The screenshot shows a GPT chat window with a table titled "2023년 7월의 일별 광고 실적" (Daily Advertising Performance for July 2023). The table has four columns: "날짜" (Date), "예상 수입 (USD)" (Estimated Revenue (USD)), and "페이지뷰" (Page Views). The data rows show dates from 2023-07-01 to 2023-07-06 with corresponding revenue and page view values. Below the table, a message states: "랜덤으로 생성된 2023년 7월의 일별 광고 실적 데이터를 담은 엑셀 파일을 생성 하였습니다. 여기에서 다운로드할 수 있습니다. [~]" (We have generated an Excel file containing randomly generated daily advertising performance data for July 2023. You can download it here. [~]).

	날짜	예상 수입 (USD)	페이지뷰
1	2023-07-01	249.82	6393
2	2023-07-02	480.29	9792
3	2023-07-03	392.8	9433
4	2023-07-04	339.46	8513
5	2023-07-05	162.41	3612
6	2023-07-06	162.4	8041

랜덤으로 생성된 2023년 7월의 일별 광고 실적 데이터를 담은 엑셀 파일을 생성 하였습니다. 여기에서 다운로드할 수 있습니다. [~]

2. 엑셀 및 구글 스프레드시트 분석

• 보고서 분석

- 보조자료의 7,8월 데이터를 GPT 프롬프트 창에 복사 + 붙여넣기 후 “두 개의 광고 보고서 파일인데, 7월과 8월을 비교해줘 특히 트렌드를 중점적으로 비교해줘” 라고 질문

7월과 8월의 광고 보고서를 비교하여 트렌드를 확인해보겠습니다. 주요 지표들을 비교하면서 각 달의 광고 실적 차이를 분석할 수 있습니다.

1. 예상 수입 (USD)

- 7월: 최소 108.23 USD에서 최대 487.96 USD까지 변동이 큼니다.
- 8월: 최소 103.68 USD에서 최대 474.69 USD까지 범위가 비슷하나, 8월에는 보다 안정적인 트렌드가 보입니다.

2. 페이지뷰

- 7월: 1161에서 9226 사이의 범위를 보입니다.
- 8월: 1098에서 9754 사이로 범위가 좁고, 평균적으로 더 높은 페이지뷰를 기록하고 있습니다.

3. 페이지 RPM (USD)

- 7월: 1.02에서 4.95 USD 사이의 RPM을 보입니다.
- 8월: 1.06에서 4.96 USD 사이로, 8월에 더 높은 RPM을 기록합니다.

4. 노출수

- 7월: 5206에서 19820 사이의 범위를 보입니다.
- 8월: 4081에서 17533 사이로, 더 넓은 범위를 보이지만 7월보다 낮은 최대치를 기록합니다.

5. 노출 RPM (USD)

- 7월: 0.52에서 2.86 USD 사이의 범위를 보입니다.
- 8월: 0.58에서 2.94 USD 사이로, 8월에 더 높은 RPM을 기록합니다.

6. 조회 가능 Active View

- 7월: 3202에서 14666 사이의 범위를 보입니다.
- 8월: 3559에서 14548 사이로, 7월보다 넓은 범위를 보이지만 7월보다 높은 최대치를 기록합니다.

7. 클릭수

- 7월: 52에서 289 사이의 범위를 보입니다.
- 8월: 59에서 295 사이로, 8월에는 더 높은 최소값을 기록하지만 전반적으로 비슷한 범위를 보입니다.

분석 요약

- 수익: 8월에는 예상 수입이 7월보다 일관되고 더 높은 평균을 보이며, 특히 RPM도 더 높습니다.
- 페이지뷰: 8월에는 평균적으로 더 높은 페이지뷰를 기록하며, 광고 페이지의 인기가 더 큼니다.
- 노출수와 클릭수: 8월에는 노출수와 클릭수가 7월보다 적지만, RPM과 예상 수입에서 더 높은 결과를 얻고 있습니다.
- 조회 가능 Active View: 8월에는 더 높은 최대치를 기록하며, 더 나은 시각적 노출이 이루어진 것으로 나타납니다.

이런 비교를 통해 7월과 8월의 광고 성과가 어떻게 변화했는지 파악할 수 있으며, 이러한 통계적 분석은 추후 광고 전략을 개선하는 데 도움이 될 수 있습니다.

2. 엑셀 및 구글 스프레드시트 분석

- 보고서 분석

- 프롬프트 입력: “8월에 예상 수입이 높아진 이유는?”



8월에 예상 수입이 높아진 주된 이유는 몇 가지 요소에 기인합니다:

1. **페이지 RPM의 증가:** 8월에는 페이지 RPM (페이지당 수익)이 7월에 비해 상승한 것으로 나타났습니다. 이는 동일한 페이지뷰에 대해 더 높은 수익을 의미합니다. 예를 들어, 광고 클릭률이나 광고 소재의 효율성이 향상되었을 수 있습니다.
2. **페이지뷰의 증가:** 8월에는 평균적으로 더 많은 페이지뷰가 기록되었습니다. 광고가 더 많은 사람들에게 노출되어 광고 수익이 증가할 수 있습니다.
3. **고가 광고 소재:** 특정 광고 소재가 더 높은 클릭률을 기록하여 더 많은 수익을 생성했을 수 있습니다. 예를 들어, 8월에는 더 많은 고가 광고 캠페인이 진행되었을 가능성이 있습니다.
4. **시장의 변화:** 광고 시장이 8월에는 더 활성화되거나, 광고주들의 광고 예산이 늘어난 경우일 수 있습니다. 이는 광고 수익에 긍정적인 영향을 미칠 수 있습니다.
5. **점검 및 최적화:** 광고 플랫폼에서 7월의 데이터를 분석하고 최적화한 결과, 8월에 더 효율적인 광고 전략이 실행되어 예상 수입이 증가했을 수 있습니다.

2. 엑셀 및 구글 스프레드시트 분석

• 보고서 분석

– 프롬프트 입력: “노출수와 클릭수가 감소한 원인은?”



노출수와 클릭수가 감소한 원인은 다양한 요소들이 복합적으로 작용한 결과일 수 있습니다. 여러 가능한 이유들을 살펴보면 다음과 같습니다:

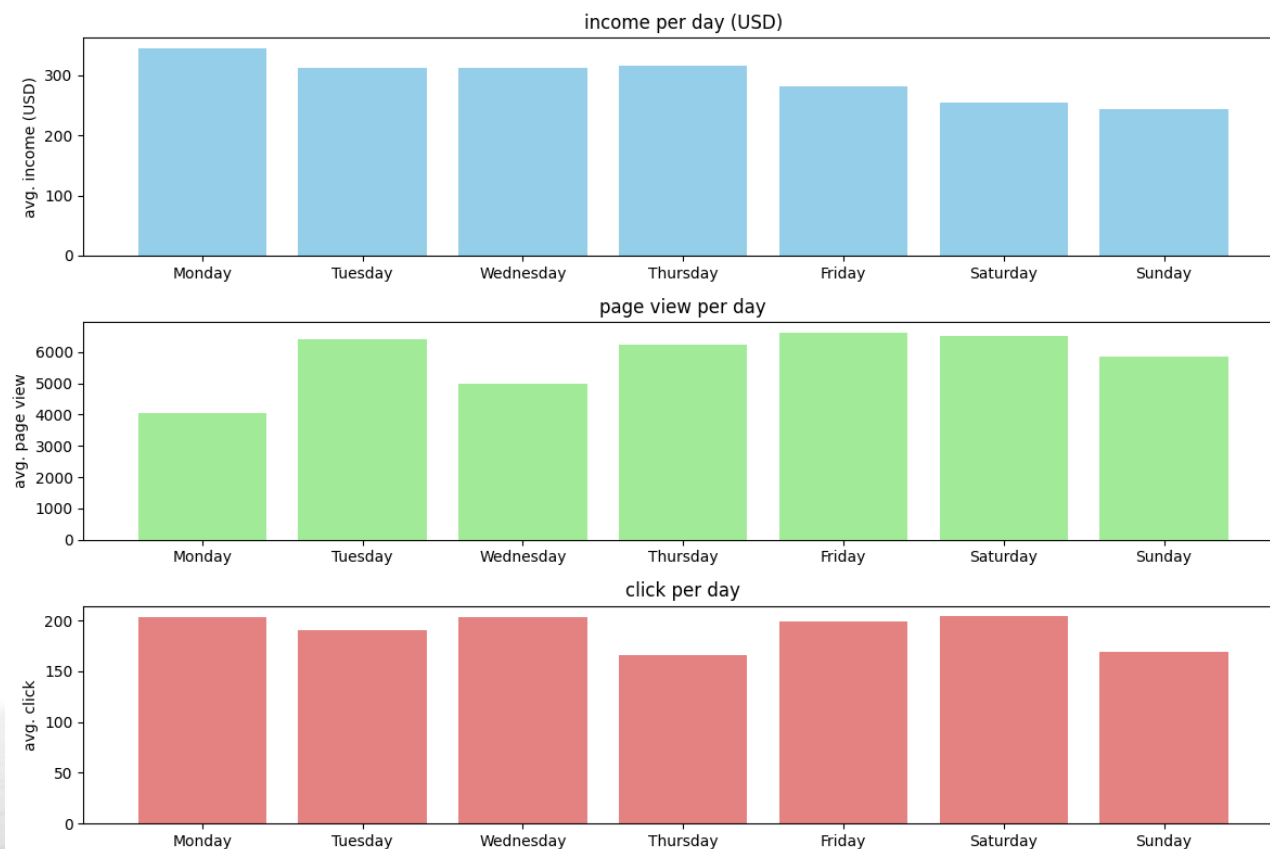
1. **계절적 요소:** 광고 산업에서는 특정 계절이나 기간에 광고 수요가 변동할 수 있습니다. 예를 들어, 휴가철이나 공휴일 기간에는 광고 활동이 줄어들 수 있습니다.
2. **경쟁 상황:** 경쟁사들의 광고 예산이 변동될 경우, 특히 경쟁이 치열한 업계에서는 노출수와 클릭수가 감소할 수 있습니다. 경쟁이 심해지면 광고 비용이 상승하거나, 광고 효과가 줄어들 수 있습니다.
3. **광고 캠페인 효율성:** 특정 광고 캠페인이나 광고 소재의 효과가 줄어든 경우에는 노출수와 클릭수가 감소할 수 있습니다. 광고 소재의 관심을 끄는 능력이나 타겟팅이 잘못된 경우에도 클릭수가 감소할 수 있습니다.
4. **플랫폼 변경:** 광고 플랫폼에서 알고리즘 변경이나 규정 변경이 있을 경우, 이로 인해 광고 노출이 줄어들거나 광고 클릭이 억제될 수 있습니다.
5. **광고 예산 조정:** 광고주가 광고 예산을 줄였거나 재조정할 경우에는 광고 노출 수와 클릭 수가 감소할 수 있습니다. 특히 예산이 축소된 경우, 특정 기간 동안 더 적은 광고가 노출될 수 있습니다.
6. **소비자 행동 변화:** 소비자의 행동이나 패턴이 변화할 경우에도 광고 성과가 변동될 수 있습니다. 예를 들어, 휴가철이나 휴일에는 온라인 활동이 줄어들거나 변동될 수 있습니다.

이러한 요소들은 각각 혹은 복합적으로 작용하여 노출수와 클릭수의 감소를 설명할 수 있습니다. 광고 성과를 분석하고 이러한 요소들을 이해하는 것이 향후 광고 전략을 개선하는 데 도움이 될 수 있습니다.

2. 엑셀 및 구글 스프레드시트 분석

- 보고서 분석

- 프롬프트 입력: “요일별 분석 및 그래프 작성바람”
- 이후 GPT가 예시 코드(Python)을 출력하였고, 아래는 출력된 코드를 실행한 결과이다. (코드는 보조자료의 example_code.ipynb)



3. 데이터 정제

- 누락값 처리

- 실습 데이터를 준비하기 위해 다음 문장을 프롬프트에 입력
- “20명의 키와 몸무게의 샘플 데이터를 만들되 일부 데이터를 누락시켜줘”
- 다음과 같은 코드로 데이터 생성

```
import pandas as pd
import numpy as np

# 샘플 데이터 생성
np.random.seed(0) # 일관된 결과를 위해 시드 설정

# 20명의 키와 몸무게 데이터 생성
heights = np.random.normal(loc=170, scale=5, size=20) # 평균 170, 표준편차 5의 정규분포로 키 생성
weights = np.random.normal(loc=65, scale=7, size=20) # 평균 65, 표준편차 7의 정규분포로 몸무게 생성

# 무작위로 일부 데이터를 NaN으로 변경하여 누락 데이터 생성
missing_indices = np.random.choice(range(20), size=5, replace=False) # 5개의 데이터를 무작위로 선택하여 누락시킴
heights[missing_indices] = np.nan
weights[missing_indices] = np.nan

# DataFrame 생성
data = {
    '키 (cm)': heights,
    '몸무게 (kg)': weights
}

df = pd.DataFrame(data)
print("누락된 데이터가 포함된 샘플 데이터:\n", df)
```

3. 데이터 정제

- 누락값 처리
 - 데이터를 정리하면 아래와 같다

키 (cm)	몸무게 (kg)
172.078173	63.764052
173.084722	75.400157
172.950088	NaN
169.671353	63.121675
166.902671	64.443863
174.481176	NaN
170.0	70.656675
170.865408	60.494079
171.393852	69.764156
169.015213	65.313068
169.156349	66.321402
168.0	69.426025
NaN	56.380684
174.156349	74.022722
174.927322	63.865409
176.984726	64.015387
167.333674	NaN
176.307756	61.600232
175.193556	61.916875
171.153036	NaN

3. 데이터 정제

- 누락값 처리

- 프롬프트 입력 : 키와 몸무게의 상관 관계를 알아보려고 하는데, 누락값을 어떻게 하면 좋을까?



키와 몸무게의 상관 관계를 분석할 때 누락된 데이터가 포함된 경우, 이를 처리하는 방법에 대해 몇 가지 고려해야 합니다. 다음은 누락된 데이터 처리 방법에 대한 몇 가지 대안입니다:

1. **데이터 제거**: 누락된 데이터가 있는 행을 제거하는 방법입니다. 이는 가장 간단하지만, 데이터의 손실이 발생할 수 있습니다. 특히 데이터가 충분히 많을 때 이 방법을 고려할 수 있습니다.
2. **평균값 대체**: 누락된 값을 전체 데이터의 평균값이나 중앙값으로 대체하는 방법입니다. 이 방법은 데이터의 왜곡을 줄이면서도 누락된 값에 대한 예측을 하게 됩니다. 하지만 평균이나 중앙값은 데이터 분포에 따라 왜곡될 가능성이 있습니다.
3. **회귀나 기타 예측 모델 사용**: 다른 변수들을 사용하여 누락된 값을 예측하는 모델을 사용하는 방법입니다. 예를 들어, 성별이나 나이와 같은 다른 변수를 사용하여 회귀 모델을 통해 누락된 값을 예측할 수 있습니다. 이 방법은 데이터의 복잡성을 고려할 때 유용할 수 있습니다.
4. **다른 데이터로 대체**: 유사한 다른 데이터나 관련된 정보를 사용하여 누락된 값을 대체하는 방법입니다. 예를 들어, 같은 사람의 다른 측정 데이터를 활용하여 누락된 값을 추정할 수 있습니다.

3. 데이터 정제

- 누락값 처리

- 프롬프트 입력: 예측 모델을 사용하는 방법에서는 누락값을 제외한 데이터에서 키와 몸무게의 상관 관계를 구한 후에 그것을 이용해서 누락값을 채우나?



예측 모델을 사용하여 누락된 데이터를 대체할 때, 일반적으로 다음과 같은 절차를 따릅니다:

1. **누락값을 제외한 데이터로 상관 관계 분석:** 먼저 누락값이 없는 데이터를 사용하여 키와 몸무게의 상관 관계를 분석합니다. 이를 통해 키와 몸무게 간의 선형 관계를 이해하고 모델링에 적용할 수 있는 기준을 마련합니다.
2. **예측 모델 구성:** 상관 관계 분석을 통해 얻은 키와 몸무게 간의 관계를 바탕으로 예측 모델을 구성합니다. 대표적으로 선형 회귀 모델이나 다른 예측 모델을 사용할 수 있습니다. 이 모델은 다른 변수들을 통해 키 또는 몸무게를 예측하는 데 사용됩니다.
3. **누락값 예측:** 예측 모델을 사용하여 누락된 데이터를 예측합니다. 이때 다른 변수들의 정보를 활용하여 누락된 값에 대한 예측을 수행합니다. 예를 들어, 성별, 나이, 기타 특성들을 고려하여 예측 모델이 적절한 키나 몸무게 값을 예측할 수 있습니다.
4. **데이터 대체:** 예측 모델을 통해 얻은 예측값을 누락된 데이터에 대체합니다.

이 절차는 주어진 데이터의 성격과 예측 모델의 복잡성에 따라 달라질 수 있습니다. 예를 들어, 단순한 예측 모델에서는 선형 회귀를 사용하여 평균값을 예측할 수 있지만, 데이터의 복잡성이 높고 다양한 변수들이 필요한 경우 복잡한 예측 모델을 구성해야 할 수도 있습니다.

3. 데이터 정제

- 누락값 처리
 - 프롬프트 입력: 채우지 말고 누락값을 제거해줘

markdown

누락된 데이터가 제거된 결과:

	키 (cm)	몸무게 (kg)
0	172.078173	63.764052
1	173.084722	75.400157
3	169.671353	63.121675
4	166.902671	64.443863
6	170.000000	70.656675
7	170.865408	60.494079
8	171.393852	69.764156
9	169.015213	65.313068
10	169.156349	66.321402
11	168.000000	69.426025
13	174.156349	74.022722
14	174.927322	63.865409
15	176.984726	64.015387
17	176.307756	61.600232
18	175.193556	61.916875

키 (cm)	몸무게 (kg)
172.078173	63.764052
173.084722	75.400157
169.671353	63.121675
166.902671	64.443863
170.0	70.656675
170.865408	60.494079
171.393852	69.764156
169.015213	65.313068
169.156349	66.321402
168.0	69.426025
174.156349	74.022722
174.927322	63.865409
176.984726	64.015387
176.307756	61.600232
175.193556	61.916875

3. 데이터 정제

- 누락값 처리

- 프롬프트 입력: 상관관계를 구해서 누락값을 채운 후 몸무게의 평균을 구해줘
- GPT가 다음과 같은 답변 생성함
- 코드는 수업 보조자료의 find_missing_value.ipynb

주어진 데이터에서 누락된 값을 채우고, 키와 몸무게의 상관관계를 계산한 후, 몸무게의 평균을 구해드릴 수 있습니다. 먼저, 데이터를 다루기 쉽도록 파이썬의 pandas 라이브러리를 사용하겠습니다.

```
python
import pandas as pd
import numpy as np

# 주어진 데이터
data_with_nan = {
    "키 (cm)": [172.078173, 173.084722, 172.950088, 169.671353, 166.902671,
    "몸무게 (kg)": [63.764052, 75.400157, np.nan, 63.121675, 64.443863, np.nan]
}

# DataFrame 생성
df = pd.DataFrame(data_with_nan)

# 누락된 값을 각 열의 평균값으로 채우기
df_filled = df.fillna(df.mean())

# 상관관계 계산
correlation = df_filled["키 (cm)"].corr(df_filled["몸무게 (kg)"])

# 몸무게의 평균 구하기
average_weight = df_filled["몸무게 (kg)"].mean()

print("상관관계:", correlation)
print("몸무게의 평균:", average_weight)
```

위 코드를 실행하면, 누락된 값이 평균값으로 채워진 데이터를 이용하여 상관관계와 몸무게의 평균이 출력됩니다. pandas의 fillna() 함수를 사용하여 누락된 값을 각 열의 평균값으로 채우고, corr() 함수를 사용하여 키와 몸무게의 상관관계를 계산하였습니다.

4. 프로그래밍

• 피즈버즈

- 피즈버즈(FizzBuzz) 문제는 프로그래밍 인터뷰에서 자주 나오는 문제 중 하나로 다음과 같은 규칙을 따른다.
 1. 1부터 N까지의 숫자를 출력한다.
 2. 그 숫자가 3의 배수라면 “피즈”를 출력한다.
 3. 그 숫자가 5의 배수라면 “버즈”를 출력한다.
 4. 그 숫자가 3과 5의 공배수라면 “피즈버즈”를 출력한다.
- GPT에 1부터 15까지의 피즈버즈를 요청한다.
- 다음과 같은 코드 및 결과 출력

```
output = []
for i in range(1, 16):
    if i % 3 == 0 and i % 5 == 0:
        output.append("피즈버즈 ")
    elif i % 3 == 0:
        output.append("피즈 ")
    elif i % 5 == 0:
        output.append("버즈 ")
    else:
        output.append(f"{i} ")

result = "".join(output).strip()
print(result)
```

plaintext

코드 복사

1 2 피즈 4 버즈 피즈 7 8 피즈 버즈 11 피즈 13 14 피즈버즈

각 숫자와 그에 해당하는 문자열("피즈", "버즈", "피즈버즈") 사이에 띄어쓰기가 포함되어 출력됩니다.

4. 프로그래밍

• 윤년 판별(1)

- 프롬프트 입력: 윤년을 판별하는 함수 `is_leap_year()`를 작성하려고 하는데, 단위 테스트를 먼저 만들고 싶다.



윤년을 판별하는 함수 `is_leap_year()`를 작성하고, 이 함수에 대한 단위 테스트를 만들어 보겠습니다. 윤년은 다음과 같은 규칙으로 정의됩니다:

1. 연도가 4로 나누어떨어지는 해는 윤년입니다.
2. 그러나 100으로 나누어떨어지는 해는 윤년이 아닙니다.
3. 다만 400으로 나누어떨어지는 해는 다시 윤년입니다.

이 규칙에 따라 윤년을 판별하는 함수와 단위 테스트를 작성해 보겠습니다.

`is_leap_year()` 함수 작성

python

코드 복사

```
def is_leap_year(year):
    """
    주어진 연도가 윤년인지 판별하는 함수

    Args:
        - year (int): 판별하고자 하는 연도

    Returns:
        - bool: 윤년이면 True, 아니면 False
    """
    if year % 4 == 0:
        if year % 100 == 0:
            if year % 400 == 0:
                return True
            else:
                return False
        else:
            return True
    else:
        return False
```

```
def is_leap_year(year):
```

```
    """
```

주어진 연도가 윤년인지 판별하는 함수

Args:

- year (int): 판별하고자 하는 연도

Returns:

- bool: 윤년이면 True, 아니면 False

```
    """
```

```
    if year % 4 == 0:
```

```
        if year % 100 == 0:
```

```
            if year % 400 == 0:
```

```
                return True
```

```
            else:
```

```
                return False
```

```
        else:
```

```
            return True
```

```
    else:
```

```
        return False
```

4. 프로그래밍

• 윤년 판별(2)

- 프롬프트 입력: 윤년을 판별하는 함수 `is_leap_year()`를 작성하려고 하는데, 단위 테스트를 먼저 만들고 싶다.

단위 테스트 작성

이제 이 함수에 대한 단위 테스트를 작성해 보겠습니다. 여기서는 `unittest` 모듈을 사용하여 간단한 단위 테스트를 구현하겠습니다.

```
python 📄 코드 복사

import unittest

class TestIsLeapYear(unittest.TestCase):

    def test_leap_year(self):
        self.assertTrue(is_leap_year(2000)) # 2000년은 윤년 (400의 배수)
        self.assertTrue(is_leap_year(2016)) # 2016년은 윤년 (4의 배수)
        self.assertTrue(is_leap_year(2400)) # 2400년은 윤년 (400의 배수)

    def test_non_leap_year(self):
        self.assertFalse(is_leap_year(1900)) # 1900년은 윤년이 아님 (100의 배수이지만 400의 배수가 아님)
        self.assertFalse(is_leap_year(2018)) # 2018년은 윤년이 아님 (4의 배수이지만 100의 배수)
        self.assertFalse(is_leap_year(2100)) # 2100년은 윤년이 아님 (100의 배수이지만 400의 배수가 아님)

    def test_edge_cases(self):
        self.assertFalse(is_leap_year(1700)) # 1700년은 윤년이 아님 (100의 배수이지만 400의 배수가 아님)
        self.assertTrue(is_leap_year(1600)) # 1600년은 윤년 (400의 배수)
        self.assertFalse(is_leap_year(0)) # 0년은 윤년이 아님 (0은 어떤

if __name__ == '__main__':
    unittest.main()
```

```
import unittest
```

```
class TestIsLeapYear(unittest.TestCase):
```

```
    def test_leap_year(self):
        self.assertTrue(is_leap_year(2000)) # 2000년은 윤년 (400의 배수)
        self.assertTrue(is_leap_year(2016)) # 2016년은 윤년 (4의 배수)
        self.assertTrue(is_leap_year(2400)) # 2400년은 윤년 (400의 배수)
```

```
    def test_non_leap_year(self):
        self.assertFalse(is_leap_year(1900)) # 1900년은 윤년이 아님 (100의 배수이지만 400의 배수가 아님)
        self.assertFalse(is_leap_year(2018)) # 2018년은 윤년이 아님 (4의 배수이지만 100의 배수)
        self.assertFalse(is_leap_year(2100)) # 2100년은 윤년이 아님 (100의 배수이지만 400의 배수가 아님)
```

```
    def test_edge_cases(self):
        self.assertFalse(is_leap_year(1700)) # 1700년은 윤년이 아님 (100의 배수이지만 400의 배수가 아님)
        self.assertTrue(is_leap_year(1600)) # 1600년은 윤년 (400의 배수)
        self.assertFalse(is_leap_year(0)) # 0년은 윤년이 아님 (0은 어떤 배수에도 해당되지 않음)
```

```
if __name__ == '__main__':
    unittest.main(argv=[''], verbosity=2, exit=False)
```

4. 프로그래밍

- 윤년 판별(3)
 - 윤년판별에 있어 어떤 학생이 다음과 같은 의견을 남김
 - GPT에 테스트및 분석을 요청

문제에서 구하고자 하는 것이 윤년판별이기 때문에 윤년인 조건만 파악하면 됩니다.
year % 400 == 0 일때만 윤년이기때문에 아래와 같은 코드로 쉽게 윤년을 판별할 수 있습니다.

```
def yoonYear(input_year) :  
    if input_year % 400 == 0:  
        print("출력연도 ", input_year, "은 윤년입니다.")  
    else :  
        print("출력연도 ", input_year, "은 평년입니다.")
```

```
yoonYear(2000)  # 출력연도 2000 은 윤년입니다.  
yoonYear(1900)  # 출력연도 1900 은 평년입니다.
```

이렇게만 해도 되는것같아요

4. 프로그래밍

- 윤년 판별(4)
 - 윤년판별에 있어 어떤 학생이 다음과 같은 의견을 남김
 - GPT에 테스트 및 분석을 요청



제시된 코드는 `input\_year`가 400의 배수일 때만 윤년으로 판별하고 있습니다. 하지만 윤년을 판별할 때는 400의 배수일 때뿐만 아니라 추가적인 조건들도 고려해야 합니다. 윤년의 정의는 다음과 같습니다:

1. 연도가 4의 배수이면서,
2. 100의 배수가 아닌 경우에는 윤년이거나,
3. 400의 배수인 경우는 무조건 윤년입니다.

따라서 제시된 코드는 모든 경우에 대해 정확한 윤년 판별을 하지 않습니다. 예를 들어, 2000년은 윤년이지만 1900년은 평년인데도 윤년으로 판별됩니다.

이를 바탕으로 수정된 함수와 이를 테스트하는 코드를 제시하겠습니다.

실행 결과

위 코드를 실행하면 각 연도에 대한 윤년 판별 결과가 정확히 출력됩니다.

plaintext

코드 복사

```
True
False
True
False
```

이렇게 수정된 코드는 주어진 조건에 맞게 모든 경우를 올바르게 판별할 수 있습니다.

```
def is_leap_year(year):
```

```
    """
```

주어진 연도가 윤년인지 판별하는 함수

Args:

- year (int): 판별하고자 하는 연도

Returns:

- bool: 윤년이면 True, 아니면 False

```
    """
```

```
    if year % 400 == 0:
```

```
        return True
```

```
    elif year % 100 == 0:
```

```
        return False
```

```
    elif year % 4 == 0:
```

```
        return True
```

```
    else:
```

```
        return False
```

테스트 코드

```
print(is_leap_year(2000)) # True (2000년은 윤년)
```

```
print(is_leap_year(1900)) # False (1900년은 평년)
```

```
print(is_leap_year(2016)) # True (2016년은 윤년)
```

```
print(is_leap_year(2100)) # False (2100년은 평년)
```

5. Agent

- Agent ?
 - 목표를 받고 계획을 세운 뒤 도구를 사용해 결과를 만드는 작업자 AI

챗봇형 사용	에이전트형 사용
사용자가 매 단계 지시합니다.	목표를 받고 필요한 단계를 선택합니다.
"이 파일 분석해줘."	"점검→분석→검증→보고서 작성까지 진행해줘."
한 번의 답변 중심	실행-관찰-수정 루프 중심
사용자는 질문자	사용자는 목표·제약·검증 기준 설계자

5 Agent

- Workflow와 Agent의 차이
 - 정해진 순서대로 실행하면 Workflow, 상황에 따라 다음 행동을 고르면 Agent

구분	Workflow	Agent
실행 순서	사람이 미리 정함	상황에 따라 모델이 선택
예외 처리	분기 규칙 필요	결과를 보고 다음 행동 결정
장점	예측 가능·관리 쉬움	복잡한 업무에 유연
주의	유연성 낮음	검증·권한·로그 필요

Agent loop = 목표 → 계획 → 도구 실행 → 관찰 → 수정 → 완료 판단

- AGENTS.md: 프로젝트 공통 규칙
 - 에이전트가 항상 따라야 하는 분석 원칙과 경로 규칙을 적는 파일.
 - AI Agent가 실행될 때마다 확인하는 상시 지침.

```
# AGENTS.md
- 원본 데이터는 수정하지 않는다.
- 분석 전 행 수, 열, 결측치, 중복을 확인한다.
- 모든 합계는 Python 코드로 계산한다.
- 모르는 원인은 추측하지 않는다.
- 결과는 output/ 폴더에 저장한다.
```


5 Agent

- SKILL.md: 반복 업무 매뉴얼
 - 특정 업무를 반복 가능하게 만드는 절차·참고자료·완료 조건.

```
---  
name: sales-analysis  
description: 월별 영업 데이터를 점검하고 KPI와 보고서를 생성한다.  
---
```

실행 절차

1. 데이터 구조와 품질 점검
2. KPI 계산
3. 차트 생성
4. 합계 계산
5. 사실·해석·가설 분리 보고

구분	역할
Prompt	이번 한 번의 업무 지시
AGENTS.md	프로젝트 공통 규칙
SKILL.md	반복 업무 절차
Tool	Python·검색·파일 등 실행 수단
Harness	규칙·권한·검증·로그를 묶은 운영 환경

Thank you

Q & A

